UNIVERSIDAD EAFIT

Escuela de Ciencias Aplicadas e Ingeniería

Departamento de Informática y Analítica

SISTEMAS OPERATIVOS – 2026-2

Evaluación parcial práctico-teórica (25%)

Seguridad de archivos mapeados con mmap() mediante intervención en la capa del kernel

Implementación de mmap_guard: un módulo de seguridad eBPF LSM para Linux 6.x

Integrantes

Juan José García

Juan Esteban Jiménez

Repositorio: **enlace del repositorio GitHub**

Referencia base: github.com/evalenciEAFIT/SO2026B (USOS/CapaKernel)

Medellín, 6 de noviembre de 2026

Contenido

1. Introducción y resumen de la solución
2. Parte I: Análisis conceptual y preguntas diagnósticas (2.1 – 2.10)
3. Parte II: Diseño de la estrategia con apoyo de IA
   3.1 Flujo de trabajo · 3.2 Prompts, alucinaciones y validación · 3.3 Arquitectura y trade-offs
4. Parte III: Implementación, presentación y argumentación
   4.1 Código · 4.2 Pruebas · 4.3 Carrera · 4.4 Overhead · 4.5 Justificación · 4.6 Limitaciones
5. Conclusiones
6. Referencias

Anexo A. Reproducción de los resultados

1. Introducción y resumen de la solución

`mmap()` proyecta un archivo en el espacio virtual de un proceso: tras el primer fallo de página, las lecturas y escrituras las resuelve la MMU **sin entrar al kernel**. Esto elimina el costo de `read()`/`write()`, pero también deja sin efecto cualquier control que dependa de interceptar esas llamadas: modificaciones persistentes vía `MAP_SHARED`, alteración o ejecución de código vía mapeos privados y escalada de permisos con `mprotect()`. El reto del parcial es reforzar la seguridad de esos archivos **desde la capa del kernel**.

Nuestra solución, **mmap_guard**, es un programa **eBPF LSM** que se engancha a los puntos de decisión del kernel donde se crean y modifican las regiones de memoria (`security_mmap_file` y `security_file_mprotect`). Aplica una política por inodo (quién puede mapear qué archivo con escritura compartida, escritura privada o ejecución) y audita cada decisión. Se carga en caliente sobre el kernel 6.6 de WSL2, sin recompilar el kernel.

Indicador	Resultado
Compilación	Clean build con `-Wall -Wextra -Werror` (programa BPF, loader, pruebas y benchmark)
Pruebas de ataque y defensa	29/29 correctas: 10 ataques bloqueados, 0 falsos positivos en accesos legítimos
Evasión por `mprotect`, enlaces simbólicos y duros, root	Bloqueada
Carrera madvise vs. escrituras (Dirty COW)	Archivo intacto en los 4 escenarios: 1,96 millones de escrituras forzadas vía /proc/self/mem contra 3,65 millones de madvise en 10 s; con defensa, el mapeo escribible se deniega
Overhead	+69 ns por mmap con archivo no protegido (≈3%) y +327 ns con archivo protegido (≈13%); sin efecto medible en fallos de página ni en read()
Kernel panics	Ninguno (el verificador eBPF garantiza seguridad de memoria)

2. Parte I: Análisis conceptual y preguntas diagnósticas

Las respuestas se apoyan en el código fuente del kernel Linux 6.6 (la versión que ejecuta nuestro entorno WSL2, `6.6.87.2-microsoft-standard-WSL2`) y, cuando aplica, en la evidencia experimental obtenida con la implementación de la Parte III. Las rutas de archivo citadas (por ejemplo `mm/mmap.c`) son relativas al árbol del kernel.

2.1 Mapeo virtual y page faults

> *¿Cuál es la secuencia exacta de eventos a nivel de kernel y MMU desde que un proceso invoca mmap() hasta que ocurre el primer fallo de página (Page Fault) por lectura o escritura en una página mapeada?*

Fase A: la llamada al sistema (crea la VMA, no carga datos).

1. La función `mmap()` de glibc ejecuta la instrucción `syscall`. La CPU cambia de CPL 3 a CPL 0 y salta a `entry_SYSCALL_64`, que guarda los registros en `struct pt_regs` y despacha a

`__x64_sys_mmap` → `ksys_mmap_pgoff()` (`mm/mmap.c`). Ahí se resuelve el descriptor a un `struct file` con `fget()`.

2. `vm_mmap_pgoff()` (`mm/util.c`) llama primero a `security_mmap_file(file, prot, flags)`. Este es el gancho LSM donde actúa nuestro programa eBPF: si algún LSM devuelve error, la llamada termina aquí y no se crea nada. Después toma el candado `mmap_write_lock(mm)`.
3. `do_mmap()` valida longitud, alineación y límites (`RLIMIT_AS`), elige la dirección virtual con `get_unmapped_area()` y traduce `PROT_*` y `MAP_*` a `vm_flags` (`VM_READ`, `VM_WRITE`, `VM_EXEC`, `VM_SHARED`, `VM_MAYREAD`, `VM_MAYWRITE`, `VM_MAYSHARE`, `VM_MAYEXEC`). También comprueba que el modo de apertura sea compatible (`FMODE_READ`/`FMODE_WRITE`) y que el montaje no sea `noexec` si se pidió `PROT_EXEC`.
4. `mmap_region()` reserva un `struct vm_area_struct` con `vm_start`, `vm_end`, `vm_pgoff`, `vm_file` (con referencia `get_file()`) y `vm_page_prot`. Luego invoca `call_mmap()` → `file->f_op->mmap()`; en ext4 es `ext4_file_mmap()`, que asigna `vma->vm_ops = &ext4_file_vm_ops` (cuyo `.fault` es `filemap_fault`). Finalmente inserta la VMA en el *maple tree* del `mm_struct` (reemplazó al árbol rojo-negro desde 6.1).
5. Se libera el candado y se devuelve la dirección virtual. **No se ha creado ninguna PTE** ni se ha leído el archivo (asignación perezosa), salvo que se pida `MAP_POPULATE` o `MAP_LOCKED`.

Fase B: el primer acceso y el fallo de página.

1. El proceso desreferencia el puntero. La MMU consulta la TLB y, al fallar, recorre las tablas desde `CR3`: PGD → P4D → PUD → PMD → PTE. La entrada final tiene `P = 0`, de modo que la CPU genera la excepción #PF (vector 14). Guarda la dirección en `CR2` y un código de error con los bits P (presente), W/R (escritura), U/S (modo usuario) e I/D (búsqueda de instrucción).
2. `asm_exc_page_fault` → `exc_page_fault()` → `do_user_addr_fault()` (`arch/x86/mm/fault.c`). Desde 6.4 se intenta primero el candado por VMA (`lock_vma_under_rcu()`) y, si no es posible, se usa `mmap_read_lock`. Se localiza la VMA y `access_error()` compara el código de error con `vma->vm_flags`: por ejemplo, una escritura en una VMA sin `VM_WRITE` termina en `SIGSEGV`.
3. `handle_mm_fault()` → `__handle_mm_fault()` crea los niveles intermedios que falten (`p4d_alloc`, `pud_alloc`, `pmd_alloc`) y llama a `handle_pte_fault()`. Como la PTE está vacía y la VMA tiene `vm_ops`, continúa en `do_fault()`.
4. **Lectura:** `do_read_fault()` aplica primero *fault-around* (`do_fault_around()`), que mapea de una vez hasta 16 páginas vecinas que ya estén en el page cache, y después llama a `vm_ops->fault` = `filemap_fault()`. Esta función busca el folio en el page cache (`filemap_get_folio`); si no está, lanza E/S con readahead (fallo mayor). Por último, `finish_fault()` → `set_pte_range()` instala la PTE a partir de `vma->vm_page_prot`: P=1, U/S=1, R/W según la VMA y NX si no hay `VM_EXEC`.
5. **Escritura:** en `MAP_PRIVATE`, `do_cow_fault()` reserva una página anónima, copia el contenido e instala una PTE escribible hacia la copia. En `MAP_SHARED`, `do_shared_fault()` invoca `vm_ops->page_mkwrite()` (el sistema de archivos reserva bloques y marca el folio como sucio) e instala una PTE escribible hacia la página del page cache.
6. Se actualizan el *reverse mapping* (`folio_add_file_rmap_ptes`) y los contadores; `iret` devuelve el control a Ring 3 y la instrucción se reejecuta, ahora con éxito. A partir de ese momento los accesos a la página **no entran al kernel**: la MMU los resuelve por hardware.

Este último punto es la razón por la cual la defensa debe actuar al crear o modificar la VMA (la Fase A), que es lo que hace nuestro programa. Nuestro benchmark confirma también el efecto de *fault-around*:

al recorrer un archivo de 16 MiB (4096 páginas) se producen alrededor de 256 fallos menores por recorrido, es decir, un fallo por cada 16 páginas (medido: 256 fallos por recorrido).

2.2 MAP_SHARED vs. MAP_PRIVATE

> *¿Cómo gestiona el subsistema de memoria virtual de Linux (struct vm_area_struct) la diferencia entre banderas MAP_SHARED y MAP_PRIVATE, y cuáles son las implicaciones de seguridad respecto a la persistencia y visibilidad cruzada entre procesos?*

La diferencia vive en `vma->vm_flags` y en `vma->vm_page_prot`, y determina qué hace el manejador de fallos ante una escritura:

Aspecto	MAP_SHARED	MAP_PRIVATE
Banderas de la VMA	`VM_SHARED | VM_MAYSHARE`. Si el archivo no se abrió con escritura, `do_mmap()` **quita `VM_MAYWRITE`**, así que `mprotect(PROT_WRITE)` tampoco podrá añadirla después.	Sin `VM_SHARED`. `VM_MAYWRITE` se conserva **aunque el fd sea `O_RDONLY`**, porque las escrituras irán a copias privadas.
PTE inicial	Escribible si `VM_WRITE`. Si el sistema de archivos rastrea páginas sucias (`vma_wants_writenotify()`), se instala protegida para que la primera escritura llame a `page_mkwrite()`.	Siempre de solo lectura (en `protection_map`, la combinación privada + escritura no da permiso de escritura): la primera escritura provoca un fallo COW.
Página física tras escribir	El folio del page cache (compartido con todos los procesos y con `read()`).	Una página anónima nueva, exclusiva del proceso (`do_cow_fault` / `wp_page_copy`).
Persistencia	Sí: el folio queda sucio y el *writeback* lo lleva a disco.	No: la copia se descarta en `munmap`/`exit`. Puede ir al swap.
Visibilidad cruzada	Inmediata para todos los procesos que mapean o leen el archivo.	Las propias escrituras son invisibles para otros. Pero las páginas aún no copiadas **siguen reflejando** cambios que otros hagan al archivo (POSIX no lo especifica).

Implicaciones de seguridad:

- `MAP_SHARED` con escritura modifica el archivo **sin pasar por `write()`**. Los controles que se aplican por escritura no lo ven: el gancho LSM `file_permission` solo actúa en las llamadas de E/S, inotify no genera `IN_MODIFY` por accesos a través del mapeo, y los antivirus o sistemas DLP que inspeccionan `write()` tampoco. El único punto de control fiable es el alta de la VMA, que es donde actúa nuestra regla `shared-write`.
- Las páginas compartidas son un canal de comunicación entre procesos: dos procesos que mapean el mismo archivo ven las escrituras del otro a velocidad de memoria, sin pasar por el kernel. Puede usarse como canal encubierto o para inyectar datos en un proceso víctima que confía en el contenido del archivo.
- `MAP_PRIVATE` protege el archivo pero no la confidencialidad: la copia privada es memoria anónima que puede ir al swap, aparecer en volcados de memoria (*core dumps*) o leerse vía `/proc/<pid>/mem` por quien tenga permiso de ptrace. Además, fue el vector de Dirty COW (pregunta 3).

2.3 Mecanismo Copy-On-Write (COW)

> *Describa el ciclo de vida de una página bajo COW. ¿En qué condiciones una falla en la gestión de banderas de protección (VM_MAYWRITE, PROT_READ, PROT_WRITE) en el kernel puede derivar en una escalada de privilegios o corrupción de archivos del sistema?*

Ciclo de vida de una página COW (mapeo privado de un archivo, o memoria anónima compartida tras `fork()`):

1. **Compartida y protegida.** La PTE apunta a la página original (del page cache, o anónima con `mapcount > 1` tras `fork`) con R/W = 0, aunque la VMA tenga `VM_WRITE`.
2. **Fallo de escritura.** La CPU genera un #PF con P=1 y W=1. `handle_pte_fault()` ve una PTE presente pero no escribible en una VMA con `VM_WRITE` y llama a `do_wp_page()`.
3. **Decisión de reutilizar o copiar.** Si la página es anónima y exclusiva del proceso (`PageAnonExclusive`, que existe desde 5.19), se reutiliza: `wp_page_reuse()` solo marca la PTE como escribible. En caso contrario, `wp_page_copy()` reserva una página nueva, copia el contenido, instala la nueva PTE (escribible y sucia) con `ptep_clear_flush()` (lo que invalida la TLB) y decrementa el `mapcount` de la original.
4. **Vida privada.** La copia es anónima y respaldada por swap. Nunca vuelve al archivo.
5. **Fin.** `munmap`, `exit` o `madvise(MADV_DONTNEED)` eliminan la PTE y liberan la copia. Un acceso posterior vuelve a fallar y, en un mapeo de archivo, se vuelve a ver el contenido original.

Cómo interactúan las banderas. `PROT_READ`, `PROT_WRITE` y `PROT_EXEC` se traducen a `VM_READ`, `VM_WRITE` y `VM_EXEC`. Los bits `VM_MAY*` (cuatro posiciones más arriba) acotan lo que `mprotect()` puede conceder después: `mprotect_fixup()` rechaza `newflags` que excedan sus `VM_MAY*` correspondientes. En un mapeo compartido de un fd sin `FMODE_WRITE`, `VM_MAYWRITE` está apagado; esa es la garantía de que nunca se podrá escribir en el archivo a través de ese mapeo.

Cuándo una falla lleva a escalada de privilegios o corrupción:

- **Dirty COW (CVE-2016-5195).** La lectura y escritura de memoria de otro proceso (`/proc/self/mem`, `ptrace`) usa `get_user_pages()` con `FOLL_WRITE | FOLL_FORCE`, lo que permite escribir en mapeos de solo lectura haciendo COW. Tras resolver el fallo COW, el código eliminaba `FOLL_WRITE` para no entrar en un bucle infinito. Si entre ese fallo y el reintento otro hilo ejecutaba `madvise(MADV_DONTNEED)`, la copia se descartaba y el reintento (ya sin `FOLL_WRITE`) encontraba la **página original del page cache**, y la escritura caía en ella. Así, un usuario sin privilegios podía modificar archivos de solo lectura como `/etc/passwd` o binarios setuid y obtener root. Se corrigió en 4.8.3 con `FOLL_COW`; en 5.19/6.0 se rediseñó con `PageAnonExclusive` (pregunta 9).
- **`VM_MAYWRITE` mal gestionado.** Si un mapeo compartido de un fd `O_RDONLY` conservara `VM_MAYWRITE`, bastaría un `mprotect(PROT_WRITE)` para escribir en un archivo sin permiso de escritura.
- **Huge Dirty COW (CVE-2017-1000405).** La misma clase de error en el camino de páginas enormes (THP, `follow_trans_huge_pmd`).
- **Familia relacionada:** Dirty Pipe (CVE-2022-0847) no es un error de COW, pero también escribía en páginas del page cache de archivos de solo lectura, en ese caso por una bandera mal inicializada en `pipe_buffer`. Muestra que **la integridad del page cache es la frontera crítica**: lo que se escribe ahí acaba en el archivo.

Nuestra prueba de carrera (sección 4.3) reproduce el patrón de Dirty COW contra un archivo 0444 de root: en 10 s, un hilo hizo 1 963 236 escrituras forzadas vía `/proc/self/mem` mientras otro ejecutaba

3 653 267 veces `madvise(MADV_DONTNEED)`, y el archivo quedó intacto (sha256 idéntico). El kernel 6.6 ya no es vulnerable.

2.4 Intercepción de llamadas en el kernel: LKM con hook en la System Call Table vs. LSM

> *Para monitorear o restringir el uso de mmap() sobre archivos sensibles, ¿cuáles son las ventajas, desventajas y viabilidad de utilizar un módulo de kernel (LKM) con hook en la System Call Table frente a un Linux Security Module (LSM) con ganchos (hooks) como file_mmap?*

> **Precisión sobre el nombre del gancho:** `file_mmap` existió hasta Linux 3.4. En 3.5 se dividió en `mmap_addr` (validación de direcciones bajas) y `mmap_file` (decisión sobre el archivo). En el kernel 6.6 el gancho correcto es `mmap_file` (`include/linux/lsm_hook_defs.h`), complementado por `file_mprotect`.

Criterio	LKM + hook en sys_call_table	LSM (gancho mmap_file / file_mprotect)
Acceso al punto de enganche	`sys_call_table` no se exporta. `kallsyms_lookup_name()` dejó de exportarse en 5.7, así que solo queda recurrir a trucos con kprobes.	API diseñada para esto, estable y documentada.
Escritura del gancho	La tabla está en memoria de solo lectura (`STRICT_KERNEL_RWX`). Hay que apagar CR0.WP, pero desde 5.3 `native_write_cr0()` fija ese bit (*CR0 pinning*): se necesita ensamblador en línea, con carreras entre CPUs.	No se modifica memoria del kernel.
Efectividad real en x86-64 moderno	**Desde 6.9 el despacho de syscalls usa un `switch` (`x64_sys_call`, mitigación de BHI)**: modificar la tabla ya no cambia qué función se ejecuta.	Independiente del mecanismo de despacho.
Cobertura	Solo la entrada por syscall: hay que enganchar `mmap`, la tabla compat de 32 bits, `mprotect`, `remap_file_pages`... **El cargador ELF (`execve`) y `shmat` llaman a `vm_mmap`/`do_mmap` internamente y no pasan por la tabla.**	`security_mmap_file()` está en `vm_mmap_pgoff()`, ruta común a la syscall, al cargador ELF y a `shmat`.
Datos disponibles	Argumentos crudos (fd, punteros de usuario). Para conocer el archivo hay que resolver el fd, con riesgo de TOCTOU.	Objetos del kernel ya resueltos (`struct file *`, `struct vm_area_struct *`), sin TOCTOU.
Estabilidad y riesgo	Alto: errores = kernel panic. Técnica típica de *rootkits*. Descargar el módulo con llamadas en curso es inseguro. Bloqueado por Secure Boot/lockdown si el módulo no está firmado.	Bajo y alineado con el diseño del kernel. Varios LSM conviven (*stacking*).
Despliegue	Carga en caliente (`insmod`) sin recompilar, pero exige compilar contra los headers exactos del kernel.	Un LSM clásico **no puede ser un módulo cargable** (`security_add_hooks` es `__init` y sus tablas son `__ro_after_init`): requiere recompilar el kernel. **BPF LSM** (5.7+) resuelve esto: carga programas en caliente, verificados por el kernel.

Conclusión. Enganchar la tabla de syscalls ya no es viable en un kernel 6.x: es inseguro, incompleto y, en x86-64 desde 6.9, inefectivo. El gancho LSM es el punto correcto. Como un LSM clásico obliga a recompilar el kernel, elegimos **BPF LSM**: conserva las ventajas del gancho LSM (cobertura, objetos resueltos, posibilidad de denegar) y se carga sin recompilar. Además, el verificador eBPF garantiza que el programa no puede colgar ni corromper el kernel. En nuestro WSL2 bastó con añadir `bpf` al parámetro de arranque `lsm=`.

2.5 Cifrado transparente en memoria

> *Si un archivo confidencial se encuentra cifrado en disco, ¿cómo debe interactuar la capa de Page Cache y el driver de sistema de archivos en el kernel para que la proyección en mmap() sea legible en el espacio de usuario autorizado sin exponer el contenido descifrado a otros procesos no privilegiados en memoria física?*

Principio: cifrar en la frontera entre el page cache y el dispositivo de bloques. `mmap()` mapea directamente los folios del page cache, así que esos folios deben contener **texto claro**: la MMU no puede descifrar en cada acceso. Por eso el sistema de archivos descifra al llenar el page cache y cifra al vaciarlo. Así lo hace fscrypt (ext4, f2fs, ubifs):

- **Lectura / fallo de página:** `filemap_fault()` → `->read_folio`/readahead envía el `bio` con el texto cifrado. Al completarse, `fscrypt_decrypt_pagecache_blocks()` descifra en el folio (AES-256-XTS por bloque, con una clave por archivo derivada por HKDF de la clave maestra) y solo entonces lo marca *uptodate* y lo mapea.
- **Escritura / writeback:** `fscrypt_encrypt_pagecache_blocks()` cifra en páginas intermedias (*bounce pages*) para no alterar el folio en claro que los procesos siguen viendo, y esas páginas son las que van a disco. Con *inline encryption* (`blk-crypto`), el controlador de almacenamiento hace el cifrado.

Cómo no exponer el texto claro a procesos no autorizados:

1. **Control en `open`/`mmap`.** El aislamiento de las tablas de páginas garantiza que un proceso solo alcanza los marcos físicos mapeados en *su* PGD. Por tanto, el texto claro del page cache solo es accesible para quien logre abrir y mapear el archivo. fscrypt exige la clave en el *keyring* del sistema de archivos para abrir el contenido, y los permisos DAC/MAC hacen el resto. Nuestro LSM añade un control específico de mmap (regla `all`), para que un usuario no autorizado no obtenga un mapeo del archivo aunque los permisos DAC lo permitan.
2. **No dejar copias en claro fuera del page cache.** Las copias COW privadas son memoria anónima que puede ir al swap en claro: hay que usar swap cifrado o `mlock()`. También conviene excluir esas regiones de los core dumps (`MADV_DONTDUMP`) y restringir `/proc/<pid>/mem` y `ptrace` (Yama `ptrace_scope`).
3. **Retirar la clave.** `FS_IOC_REMOVE_ENCRYPTION_KEY` borra la clave y expulsa del page cache los inodos y folios en claro.
4. **Ataques físicos.** El contenido en claro está en RAM, así que contra ataques de arranque en frío (*cold boot*) o DMA se requiere cifrado de memoria por hardware (AMD SME/SEV, Intel TME/TDX) e IOMMU.

Limitaciones. El page cache es único por inodo: si un proceso autorizado ya cargó el folio en claro, otro proceso con permiso DAC sobre el archivo lo puede leer con `read()`. El propio diseño de fscrypt advierte que no es un mecanismo de control de acceso entre usuarios del sistema en ejecución. dm-crypt (cifrado de disco completo) solo protege ante ataques sin conexión. Un diseño apilado como

eCryptfs mantiene dos page caches (cifrado abajo, claro arriba). La solución completa combina fscrypt con un LSM que controle la apertura y el mapeo; nuestro programa cubre la parte de mapeo.

2.6 Sincronización e integridad (msync y fsync)

> *¿Cómo garantiza el kernel la consistencia de páginas marcadas como sucias (dirty pages) en memoria antes de ser persistidas en disco, y cómo puede un atacante evadir filtros de seguridad si los datos no son inspeccionados en el momento de la escritura en memoria?*

Seguimiento de páginas sucias en mapeos compartidos:

1. En la primera escritura, `do_shared_fault()`/`wp_page_shared()` invocan `->page_mkwrite()`: el sistema de archivos reserva bloques (o falla con `SIGBUS` si no hay espacio) y espera a que termine un writeback en curso (*stable pages*). Después la PTE queda escribible y el folio se marca sucio (`folio_mark_dirty`).
2. Las escrituras siguientes las hace el hardware: la MMU pone el bit **D (dirty)** de la PTE. El kernel no las ve.
3. El writeback (hilos de *flusher*, por antigüedad `dirty_expire_centisecs` = 30 s o por proporción `dirty_background_ratio`; `balance_dirty_pages()` frena a los procesos que ensucian demasiado) escribe el folio. Antes de hacerlo, `folio_mkclean()` recorre el *rmap* y **protege contra escritura y limpia todas las PTE** que lo mapean. Así, la próxima escritura vuelve a provocar un fallo y vuelve a marcar el folio, sin perder modificaciones.
4. `msync(MS_SYNC)` y `fsync()` → `vfs_fsync_range()` → `file_write_and_wait_range()` y el *commit* del journal: retornan solo cuando los datos (y los metadatos, en `fsync`) son durables. `MS_ASYNC` no hace nada desde 2.6.19, porque las páginas ya están rastreadas. `fdatasync()` omite los metadatos no esenciales.

Cómo se evaden los filtros que no inspeccionan en el momento de la escritura:

- **Invisibilidad.** Los filtros enganchados a `write()` (LSM `file_permission`, fanotify, inotify `IN_MODIFY`/`IN_CLOSE_WRITE`, antivirus, DLP) nunca ven los almacenamientos hechos a través del mapeo.
- **TOCTOU.** Si se inspecciona en `msync` o en el writeback, el atacante sigue escribiendo justo después de la inspección. Entre dos `folio_mkclean` solo hay **un** fallo de página por ciclo, no uno por escritura.
- **Atribución perdida.** El writeback ocurre en un hilo del kernel, sin relación con el proceso que escribió.
- **Manipulación transitoria.** Se puede modificar el archivo en memoria (lo ven de inmediato los demás procesos que lo mapean o lo leen) y restaurarlo antes del writeback. Sin rastro en disco.

Mitigación. Decidir en el alta de la VMA (nuestra regla `shared-write`: si el mapeo escribible no existe, no hay nada que evadir). Para datos inmutables, verificar integridad con fs-verity, dm-verity o IMA en modo *appraisal*. Para auditoría, rastrear `page_mkwrite` o `folio_mark_dirty` con eBPF (pregunta 7).

2.7 Control de accesos con eBPF

> *¿Cómo se puede emplear eBPF (Extended Berkeley Packet Filter) y kprobes/fexit para auditar accesos indebidos o patrones anómalos de lectura/escritura sobre regiones mapeadas con mmap() sin recompilar el núcleo del sistema operativo?*

eBPF permite cargar en caliente programas que el **verificador** valida (terminación acotada, accesos a memoria seguros, tipos comprobados con BTF) y que luego se compilan JIT a código nativo. Requiere `CONFIG_BPF_SYSCALL`, BTF (`/sys/kernel/btf/vmlinux`) y kprobes; nuestro kernel WSL2 los trae activos. Con CO-RE, un mismo objeto funciona en distintas versiones del kernel. Puntos de enganche útiles para regiones mapeadas:

Punto de enganche	Qué se audita	Puede bloquear
`fentry`/`fexit` de `vm_mmap_pgoff`, `security_mmap_file`	Quién mapea qué archivo con qué `prot`/`flags`; `fexit` ve además el valor de retorno (dirección o error).	No
`kprobe`/`kretprobe` de `do_wp_page`, `wp_page_copy`	Tormentas de COW: muchas copias por segundo sobre un mismo archivo.	No
`kprobe` de `filemap_page_mkwrite` / `folio_mark_dirty`	Primera escritura por ciclo de writeback en mapeos compartidos.	No
tracepoint `exceptions:page_fault_user`	Dirección y código de error de cada #PF de usuario; permite cruzar con rangos de VMAs protegidas guardados en un mapa.	No
tracepoints `syscalls:sys_enter_madvise`, `sys_enter_mprotect`	`MADV_DONTNEED` en bucle o cambios W/X: señales del patrón Dirty COW.	No
`fentry` de `mem_rw` / `access_remote_vm`	Escrituras forzadas vía `/proc/<pid>/mem`.	No
`lsm/mmap_file`, **`lsm/file_mprotect`** (nuestra solución)	Todo lo anterior al nivel de la VMA, con decisión.	**Sí** (`-EPERM`)

Patrón de detección de anomalías. Un mapa `LRU_HASH` indexado por PID cuenta eventos (madvise, COW, escrituras por `/proc/<pid>/mem`) en una ventana de tiempo (`bpf_ktime_get_ns`). Si se supera un umbral, el programa emite un evento por *ring buffer* y un proceso en espacio de usuario alerta o mata al proceso. Ejemplo con bpftrace para registrar mapeos escribibles compartidos:

```
sudo bpftrace -e 'kprobe:security_mmap_file
  /arg0 != 0 && (arg1 & 0x2) && (arg2 & 0x1)/     // security_mmap_file(file, prot, flags)
  { printf("%s pid=%d mapea W+SHARED ino=%lu\n", comm, pid,
           ((struct file *)arg0)->f_inode->i_ino); }'
```

Límites. Las sondas de traza observan, pero no pueden impedir la operación: solo `bpf_override_return` sobre funciones marcadas `ALLOW_ERROR_INJECTION`, lo cual no es apto para seguridad. Por eso la aplicación de la política se hace con programas LSM. Además, ningún mecanismo ve las cargas y almacenamientos individuales una vez instalada la PTE: solo los fallos. Observar cada acceso exigiría mantener las PTE no presentes (como hacen `userfaultfd` o los depuradores), con un costo de un fallo por acceso. Los kprobes dependen de funciones internas que cambian entre versiones; fentry/fexit con BTF es más eficiente (trampolín directo) y tipado.

2.8 Evasión de permisos POSIX

> *Si un proceso abre un descriptor de archivo con permisos de solo lectura (O_RDONLY), ¿puede bajo ciertas configuraciones de kernel mapearlo con PROT_READ | PROT_WRITE y MAP_PRIVATE?*

> *Explique por qué el kernel lo permite y qué riesgos de fuga de información o alteración temporal de datos implica.*

Sí, y no depende de la configuración: lo permite cualquier kernel Linux, por diseño y conforme a POSIX. En `do_mmap()` (`mm/mmap.c`) las comprobaciones de modo de apertura son distintas según el tipo de mapeo:

```
case MAP_SHARED:  /* (resumido de mm/mmap.c, Linux 6.6) */
    if ((prot & PROT_WRITE) && !(file->f_mode & FMODE_WRITE))
        return -EACCES;                 /* escribir el archivo exige fd escribible */
    if (!(file->f_mode & FMODE_WRITE))
        vm_flags &= ~(VM_MAYWRITE | VM_SHARED);
    fallthrough;
case MAP_PRIVATE:
    if (!(file->f_mode & FMODE_READ))
        return -EACCES;                 /* privado solo exige poder leer */
```

El razonamiento es que en `MAP_PRIVATE` las escrituras **nunca llegan al archivo**: van a copias COW anónimas. Escribir en la copia equivale a leer el archivo a un buffer propio y modificar ese buffer, algo que el proceso ya podía hacer con `read()`. Es el mecanismo que usa el cargador dinámico para aplicar relocalizaciones sobre las secciones de datos de las bibliotecas.

Evidencia experimental. En nuestra fase de línea base, el usuario `nobody` abrió un archivo con `O_RDONLY`, lo mapeó `PROT_READ|PROT_WRITE, MAP_PRIVATE` y escribió en él. Resultado: "copia privada modificada, archivo intacto (COW)". Con la defensa activa y la regla `private-write`, el mismo intento devuelve `EPERM`, tanto en `mmap()` como por la vía `mmap(PROT_READ)` + `mprotect(PROT_WRITE)` (29/29 pruebas OK).

Riesgos:

- **Alteración temporal de datos.** El proceso ve una versión modificada de un archivo (configuración, tabla de licencias, código de una biblioteca) sin cambiarlo en disco. Si después consigue `PROT_EXEC` sobre esa región, tiene inyección de código W→X; por eso nuestra política también controla `exec`. Además, un verificador de integridad que inspeccione la memoria del proceso en lugar del archivo puede ser engañado.
- **Fuga de información.** Las copias privadas son memoria anónima: pueden ir al swap sin cifrar, aparecer en *core dumps* y leerse vía `/proc/<pid>/mem`. Antes de la copia, el mapeo privado sigue mostrando los cambios que otros procesos hacen al archivo, lo que expone escrituras ajenas en curso.
- **Superficie de ataque.** Este escenario exacto (archivo de solo lectura con mapeo privado) es el que Dirty COW convirtió en escritura real sobre el archivo. Un error del kernel en la frontera COW transforma un permiso "inocuo" en una corrupción del sistema.

2.9 Mitigación de ataques de carrera (race conditions)

> *¿Cómo previene el kernel actual condiciones de carrera concurrentes entre madvise(MADV_DONTNEED) y escrituras sobre páginas mapeadas privadas?*

Intervienen tres mecanismos complementarios:

1. **Candados de tabla de páginas y revalidación.** `madvise(MADV_DONTNEED)` → `madvise_dontneed_free()` → `zap_page_range_single()` toma `mmap_read_lock` (puede

correr en paralelo con los fallos de página) y, para cada tabla, el **PTL** (*page table lock*, por página de tabla con *split ptlocks*). El manejador de fallos prepara la página fuera del candado, toma el mismo PTL y, antes de instalar, comprueba con `pte_same()` que la PTE no cambió; si cambió, descarta el trabajo y reintenta. Nunca se instala una PTE basada en un estado obsoleto.

2. **Liberación diferida con `mmu_gather`.** El *zap* acumula las páginas a liberar y solo las devuelve al asignador después de invalidar la TLB en todas las CPU (`tlb_finish_mmu`). Así, ningún hilo puede seguir escribiendo en una página ya liberada a través de una entrada TLB obsoleta.
3. **Corrección semántica de GUP con FOLL_FORCE** (la raíz de Dirty COW). Desde 5.19/6.0, el parche *"mm/gup: fix FOLL_FORCE COW security issue and remove FOLL_COW"* hace que `can_follow_write_pte()` permita una escritura forzada sobre una PTE no escribible **solo si la página es anónima y exclusiva** (`PageAnonExclusive`) en una VMA privada con `VM_MAYWRITE`. Si `MADV_DONTNEED` eliminó la copia, la búsqueda falla, GUP vuelve a provocar el fallo COW (creando otra copia exclusiva) y reintenta. **Nunca** se devuelve la página del page cache para escribir. Ya no existe el "olvidar `FOLL_WRITE`" que causaba el error.

A esto se suman los candados por VMA (6.4) para los cambios estructurales (`vma_start_write`). `MADV_DONTNEED` no cambia la estructura de la VMA, solo sus PTE.

Evidencia experimental (`tests/run_race.sh`, 10 s por escenario, `nobody` contra un archivo `root:root 0444`): (1) sin defensa, `/proc/self/mem`: 1 963 236 escrituras contra 3 653 267 `madvise`, archivo **intacto**; (2) sin defensa, escritura directa COW: 869 691 015 escrituras contra 4 104 992 `madvise`, archivo **intacto**; (3) con defensa: `mmap(PROT_WRITE, MAP_PRIVATE)` **denegado con EPERM** (regla `private-write`), el ataque ni siquiera empieza; (4) con defensa, `/proc/self/mem`: 2 068 280 escrituras, archivo **intacto** gracias al kernel. Las escrituras forzadas no fallaron (`errores_escritura=0`) porque aterrizan en copias anónimas exclusivas, nunca en el page cache.

2.10 Aislamiento de memoria y paginación

> *¿Qué papel juegan las estructuras de descriptores de página (struct page), tablas de páginas (PGD, P4D, PUD, PMD, PTE) y los bits de protección de la MMU (NX, R/W, U/S) en la contención de inyecciones de código malicioso en memoria mapeada?*

`struct page` / `struct folio`. Hay un descriptor por marco físico (en `vmemmap`). Contiene `flags` (`PG_locked`, `PG_dirty`, `PG_uptodate`, `PG_writeback`, `PG_anon_exclusive`), `_refcount`, `_mapcount`, `mapping` (el `address_space` del archivo, o el `anon_vma` con el bit bajo activado) e `index`. Con esa información el *reverse mapping* encuentra **todas** las PTE que apuntan a un marco. Eso permite quitarle la escritura en todos los procesos a la vez (COW, `folio_mkclean`), desmapearlo o decidir si una página anónima es exclusiva. Sin esa contabilidad no habría COW seguro.

Jerarquía de tablas. Cada proceso tiene su propio PGD (`mm->pgd`), que se carga en `CR3` en cada cambio de contexto (con PCID para no vaciar la TLB). La traducción recorre PGD → P4D (solo con paginación de 5 niveles, LA57) → PUD → PMD → PTE. **El aislamiento entre procesos es estructural:** un proceso solo puede direccionar los marcos que aparecen en *sus* tablas. La memoria de otro proceso o del kernel simplemente no tiene traducción. Con KPTI (mitigación de Meltdown), la tabla de usuario ni siquiera contiene la mayor parte del kernel.

Bit (x86-64)	Significado	Rol contra la inyección de código
P (bit 0)	Página presente	Si P=0, cualquier acceso entra al kernel (#PF): es lo que permite carga perezosa, COW y control.
R/W (bit 1)	0 = solo lectura	Impide escribir código o datos protegidos. Los mapeos COW y de solo lectura lo tienen en 0; CR0.WP lo hace cumplir **incluso en Ring 0** (por eso apagarlo para enganchar la tabla de syscalls es peligroso).
U/S (bit 2)	0 = solo supervisor	El código de usuario no puede leer ni ejecutar memoria del kernel. Con SMEP/SMAP (CR4), el kernel tampoco ejecuta ni lee memoria de usuario accidentalmente (contra ataques *ret2usr*).
A / D (bits 5, 6)	Accedida / sucia	Base del writeback y de la detección de modificaciones (pregunta 6).
PKEY (bits 59–62)	Clave de protección (MPK)	Permite revocar el acceso a regiones dentro del mismo proceso sin `mprotect` (`pkey_mprotect`, registro PKRU).
NX / XD (bit 63)	No ejecutable (requiere EFER.NXE)	Una búsqueda de instrucción en una página NX produce #PF con I/D=1 → `SIGSEGV`. Es la base de **W^X**: los datos inyectados no se pueden ejecutar.

Los permisos efectivos combinan todos los niveles: R/W y U/S deben estar permitidos en cada nivel y NX en cualquier nivel prohíbe ejecutar. **Las PTE heredan sus bits de `vma->vm_page_prot`**, que se deriva de `vm_flags`. Por eso controlar `VM_WRITE` y `VM_EXEC` en el alta (`mmap`) y en la modificación (`mprotect`) de la VMA determina los bits de hardware de **todas** las páginas que se instalen después.

Nuestra regla `exec` impide que una región de un archivo protegido tenga `VM_EXEC`; en consecuencia, todas sus PTE tendrán NX=1 y el hardware bloqueará cualquier intento de ejecutar desde ella. Las reglas `write` y `shared-write` hacen lo análogo con R/W. La comprobación en `mprotect` cierra la secuencia clásica de inyección "escribir datos → cambiar a ejecutable → saltar" (W→X).

3. Parte II: Diseño de la estrategia con apoyo de inteligencia artificial

Usamos un asistente de IA (Claude, en modo Cowork, con acceso a la carpeta del proyecto) como acelerador en todo el ciclo. Las decisiones de arquitectura, la ejecución en el entorno real y la validación final quedaron a cargo del equipo. El registro completo de interacciones está en `bitacora_ia/BITACORA_PROMPTS.md`; esta sección lo resume.

3.1 Flujo de trabajo asistido por IA

Fase	Rol de la IA	Rol del equipo	Artefacto resultante
Exploración arquitectónica	Desglosar el enunciado contra la rúbrica. Comparar las opciones LKM/syscall table, LKM con kprobe/ftrace, LSM clásico y BPF LSM. Diseñar el diagnóstico del kernel (`/proc/config.gz`, BTF, securityfs). Identificar ganchos y estructuras: `security_mmap_file`, `security_file_mprotect`, `vm_area_struct`, `vm_flags`.	Ejecutar el diagnóstico en WSL2 y elegir la opción A (BPF LSM). Configurar `.wslconfig` y verificar que `bpf` aparezca en `/sys/kernel/security/lsm`.	Plan de trabajo, `docs/entorno/.wslconfig`, decisión de arquitectura.
Generación y verificación de código C de bajo nivel	Escribir el programa eBPF LSM, el loader con libbpf/skeleton, el parser de políticas y el Makefile. Compilar contra el BTF de un kernel 6.x con `-Werror` antes de entregar.	Compilar en WSL contra el BTF del kernel en ejecución, revisar el código y validar la ejecución real.	`src/`, `Makefile`. Clean build verificado (cero warnings).
Casos de prueba y PoC controlados	Diseñar la matriz de 13 ataques (incluye intentos de evasión vía `mprotect`, enlaces simbólicos y duros, y root sin autorización), la prueba de carrera con el patrón de Dirty COW y el benchmark de overhead.	Ejecutar las pruebas (requieren root y BPF LSM activo) y analizar los resultados.	`tests/`, `bench/`, `results/`.
Documentación	Redactar los borradores de las respuestas teóricas con referencias al código del kernel 6.6.	Revisar, contrastar con el código fuente y preparar la sustentación.	Este informe y el README.

3.2 Metodología de ingeniería de prompts y verificación crítica

3.2.1 Prompts estructurados utilizados

Cada interacción se documentó con el contexto entregado, el rol esperado del modelo, las restricciones y el resultado esperado. Las peticiones se hicieron en lenguaje natural dentro de una conversación continua, de modo que el contexto incluye los archivos y resultados de los pasos anteriores.

#	Contexto	Rol	Restricción	Resultado esperado → obtenido
1	PDF del parcial completo.	Asistente de planeación técnica.	Separar lo que hace la IA de lo que depende del equipo; respetar la rúbrica.	Plan de desarrollo → plan por fases, comparación LKM vs. LSM vs. BPF LSM y riesgos técnicos (CR0, `kallsyms_lookup_name`, despacho por `switch` en 6.9).
2	Repositorio del curso y Linux disponible solo en WSL2.	Ingeniero de entorno.	Sin VM adicional; no recompilar el kernel.	Viabilidad del entorno → script de diagnóstico; resultado: `CONFIG_BPF_LSM=y` y BTF disponibles, pero `bpf` inactivo.

#	Contexto	Rol	Restricción	Resultado esperado → obtenido
3	Salida del diagnóstico (BPF LSM compilado, inactivo).	Ingeniero de entorno.	Configuración reversible; no tocar el kernel.	Activar BPF LSM → `.wslconfig` con `kernelCommandLine=lsm=...,bpf`; verificado en `/sys/kernel/security/lsm`.
4	Entorno listo: kernel 6.6, libbpf 1.3, clang 18.	Desarrollador de sistemas C/eBPF.	Ganchos LSM reales, clean build, política por inodo, cubrir el bypass por `mprotect`, sin kernel panics.	Programa, loader, Makefile y pruebas → 29/29 pruebas OK en WSL.
5	Implementación validada; entrega en pareja el mismo día.	Ingeniero de pruebas y redactor técnico.	Pruebas de carrera y overhead reproducibles con perf/strace/ftrace; respuestas con estructuras internas del kernel.	Prueba de carrera, benchmark e informe → este documento y `results/`.

3.2.2 Análisis de alucinaciones y errores técnicos detectados

Distinguimos entre errores **efectivamente cometidos** por el modelo y detectados en la validación, y verificaciones preventivas sobre errores típicos de los LLM en programación de kernel. Solo los primeros se reportan como hallazgos.

ID	Qué propuso la IA	Cómo se detectó	Verificación / fuente	Corrección
H1	Supuso que el kernel de WSL2 podría no traer BPF LSM y propuso una VM como alternativa.	Diagnóstico en la máquina real.	`/proc/config.gz`: `CONFIG_BPF_LSM=y`. Lo que faltaba era `bpf` en `CONFIG_LSM` (no activo en el arranque).	Activarlo con `kernelCommandLine` en `.wslconfig`.
H2	Banderas BPF `-Wall -Wextra -Werror` sin excepciones.	Error de compilación: la macro `BPF_PROG` genera un parámetro `ctx` sin uso.	`bpf/bpf_tracing.h` (libbpf 1.3), definición de `BPF_PROG`.	`-Wno-unused-parameter` solo para el objeto BPF; el resto sigue con `-Werror`.
H3	Leer campos del kernel con `BPF_CORE_READ()`.	Revisión: usa `bpf_probe_read_kernel()`, que el kernel deshabilita en *lockdown* de confidencialidad.	`kernel/trace/bpf_trace.c`: `security_locked_down(LOCKDOWN_BPF_READ_KERNEL)`.	Accesos directos a punteros BTF (`file->f_inode->i_ino`, `bpf_get_current_task_btf()`), validados por tipo.
H4	Validar la carga del programa en el entorno de la IA (kernel 6.18 con BPF LSM).	Incluso un programa LSM trivial fallaba con `EPERM`.	Restricción del sandbox; no dice nada del código.	La validación en ejecución se hizo solo en WSL (evidencia en `results/`).
H5	Ejemplo de bpftrace con índices de argumentos del gancho LSM (`reqprot`, `prot`, `flags`) aplicados a `security_mmap_file`.	Revisión del borrador de la pregunta 7.	`security_mmap_file(file, prot, flags)` tiene 3 argumentos (`security/security.c`).	Índices corregidos: `arg1` = prot, `arg2` = flags.

ID	Qué propuso la IA	Cómo se detectó	Verificación / fuente	Corrección
H6	Medición con ftrace (`function_graph` + `set_graph_function` + filtro por PID) en el script de benchmark.	En WSL2 el trace salió vacío (solo la cabecera).	Causa no diagnosticada antes de la entrega; perf y strace sí funcionaron.	El costo del gancho se midió por diferencia de tiempos S1/S2 frente a S0; ftrace queda como trabajo pendiente.
Enunciado	El enunciado nombra el gancho `file_mmap`.	La IA lo señaló como nombre obsoleto.	`include/linux/lsm_hook_defs.h` (6.6): `mmap_file` y `mmap_addr` (división en 3.5).	El código usa `SEC("lsm/mmap_file")`.

Verificaciones preventivas (errores típicos de LLM en código de kernel que revisamos explícitamente y **no** aparecen en nuestro código):

- Deshabilitar el bit WP de CR0 con `write_cr0()` para parchear `sys_call_table`: inefectivo desde 5.3 (*CR0 pinning*) e inseguro sin control de concurrencia (otra CPU podría ejecutar con WP=0, y la ventana no es atómica respecto a interrupciones).
- Usar `kallsyms_lookup_name()` desde un módulo: no se exporta desde 5.7 (no tiene `EXPORT_SYMBOL`).
- Enganchar `sys_call_table` en x86-64 ≥ 6.9: el despacho ya no consulta la tabla.
- Registrar un LSM clásico desde un módulo cargable: `security_add_hooks()` es `__init`. Por eso optamos por BPF LSM.

3.2.3 Procedimiento de validación manual y compilación

1. **Contra el kernel en ejecución:** `bpftool btf dump file /sys/kernel/btf/vmlinux format c > build/vmlinux.h` genera los tipos exactos del kernel 6.6.87.2 de WSL2. CO-RE ajusta los desplazamientos de campo al cargar (`libbpf: relo ... patched insn`).
2. **Compilación limpia:** `clang -target bpf -Wall -Wextra -Werror` para el objeto BPF y `gcc -std=gnu11 -Wall -Wextra -Werror` para el loader, las pruebas y el benchmark. Resultado en WSL: cero warnings.
3. **Verificador del kernel:** al cargar, el verificador eBPF rechaza cualquier acceso a memoria no comprobado, bucle no acotado o tipo incorrecto. Que el programa cargue en el kernel 6.6 es en sí una validación formal de seguridad de memoria.
4. **Contraste con el código fuente:** las firmas de los ganchos (`lsm_hook_defs.h`), las banderas (`VM_SHARED` = 0x8, `MAP_TYPE` = 0x0f) y la semántica de `do_mmap()` se comprobaron en el árbol 6.6.
5. **Pruebas diferenciales:** cada ataque se ejecuta primero sin defensa (debe funcionar) y luego con defensa (debe fallar). Así cada denegación se atribuye al LSM y no a los permisos DAC (los archivos de prueba son 0666).
6. **Recuperación:** si algo falla, solo se reinicia la VM de WSL (`wsl --shutdown`), no Windows. La protección desaparece al terminar el loader.

3.3 Estrategia arquitectónica seleccionada

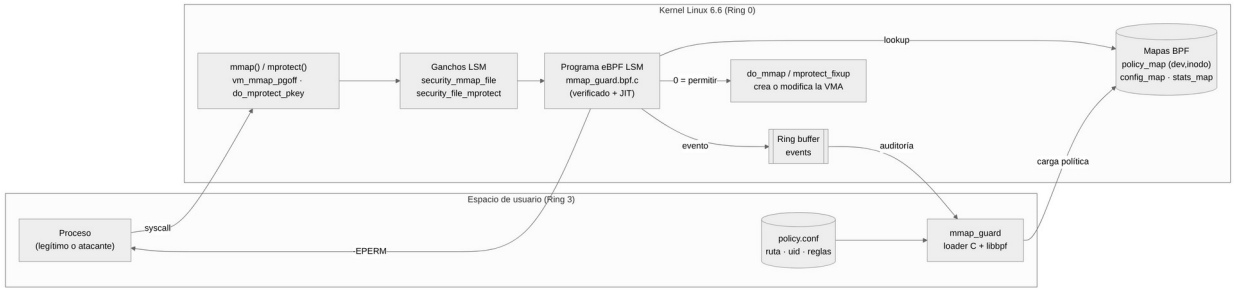


Figura 1. Arquitectura de mmap_guard: loader en espacio de usuario, mapas BPF y programa eBPF LSM en los ganchos de mmap y mprotect.

Descripción. La solución, `mmap_guard`, tiene tres piezas:

- **Programa eBPF LSM** (`src/bpf/mmap_guard.bpf.c`): se engancha a `lsm/mmap_file` y `lsm/file_mprotect`. Para cada solicitud obtiene `(s_dev, i_ino)` del inodo, busca la política en un mapa hash (tiene un costo O(1) y, si el archivo no está protegido, el programa termina ahí). Luego evalúa las reglas contra `prot` y el tipo de mapeo (y `vm_flags` en mprotect), exime al UID efectivo autorizado y devuelve `0` o `-EPERM`. Cada decisión relevante se publica en un *ring buffer*.
- **Mapas BPF:** `policy_map` (hash, clave dev+inodo), `config_map` (modo bloqueo o auditoría), `stats_map` (contadores por CPU, sin contención) y `events` (*ring buffer* de 256 KiB).
- **Loader** (`src/loader/`): lee `policy.conf`, traduce rutas a inodos (`stat()` sigue enlaces simbólicos), convierte el `dev_t` de glibc al formato interno del kernel (`major << 20 | minor`), carga el programa con el *skeleton* de libbpf, llena los mapas, engancha los programas e imprime o registra la auditoría en JSON Lines.

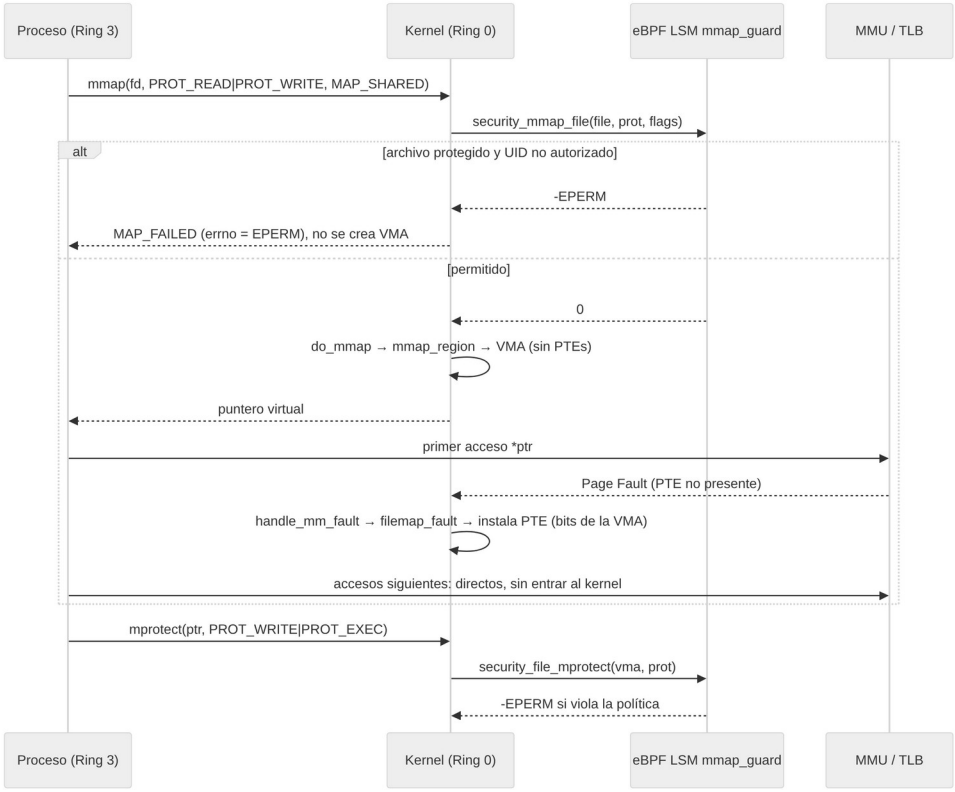


Figura 2. Secuencia: la decisión se toma al crear o modificar la VMA; los accesos posteriores los resuelve la MMU.

Matriz de compensaciones (trade-offs). Los valores de rendimiento de nuestra opción provienen de la sección 4.4.

Alternativa	Costo en mmap/mprotect	Costo por page fault	Profundidad de inspección	Cobertura	Estabilidad / despliegue
LD_PRELOAD (usuario)	Muy bajo	Nulo	Argumentos de la llamada	Eludible: binario estático, syscall directa, `unsetenv`	Trivial, pero no es seguridad
LKM + sys_call_table	Bajo	Nulo	Argumentos crudos (TOCTOU)	Incompleta (ELF, shmat, compat). Inefectiva en ≥ 6.9	Riesgo de panic; CR0 pinning
LKM + kprobe/ftrace	Bajo a medio (trampolín)	Nulo, si no se engancha el fault	Objetos del kernel	Buena, pero depende de funciones internas	Frágil entre versiones; requiere headers del kernel
LSM clásico	Muy bajo (llamada directa)	Nulo	Objetos del kernel	Completa (todas las rutas de `vm_mmap`)	Estable, pero exige **recompilar el kernel**
BPF LSM (elegida)	**Bajo** (+69 a +327 ns por mmap)	**Nulo** (no toca el fault)	Objetos del kernel + política dinámica	Completa en mmap/mprotect	**Verificador**: sin panics; en caliente

Alternativa	Costo en mmap/mprotect	Costo por page fault	Profundidad de inspección	Cobertura	Estabilidad / despliegue
Instrumentar handle_mm_fault / PTEs no presentes	Bajo	**Muy alto** (cada acceso inicial o todos)	Página por página (podría cifrar o inspeccionar contenido)	Ve los accesos	Complejo; degrada todo el sistema
eBPF solo de traza	Bajo	Bajo a medio, si engancha faults	Observación	Detecta pero **no bloquea**	Seguro, en caliente

Elegimos controlar la **creación y modificación de la VMA** en lugar del page fault. Este enfoque tiene costo cero en la ruta caliente (los fallos y los accesos posteriores) y garantiza, a través de los bits que hereda cada PTE, que ninguna página del archivo protegido llegue a ser escribible o ejecutable para un sujeto no autorizado. A cambio, no inspecciona el **contenido** página a página: el cifrado o la inspección por página requerirían instrumentar el fallo, con un costo alto (ver 4.5).

4. Parte III: Implementación, presentación y argumentación

4.1 Entregables de código

Archivo	Contenido
`src/common.h`	Estructuras compartidas kernel/usuario: `file_key` (dev, inodo), `file_policy`, `guard_config`, `mmap_event`, y las banderas `POL_*`.
`src/bpf/mmap_guard.bpf.c`	Programa eBPF LSM: ganchos `lsm/mmap_file` y `lsm/file_mprotect`, mapas, decisión y auditoría.
`src/loader/mmap_guard.c`	Loader (libbpf + skeleton): opciones `-c -a -v -l -d -s`, carga, enganche, consumo del ring buffer y contadores.
`src/loader/policy.c/.h`	Parser de `policy.conf`: ruta → (dev, inodo), reglas, UID autorizado.
`tests/mmap_probe.c`	Ejecuta un tipo de acceso concreto (7 modos) e informa si fue permitido o denegado.
`tests/run_tests.sh`	Matriz de ataque y defensa: línea base, bloqueo, integridad y auditoría.
`tests/race_cow.c`, `tests/run_race.sh`	Carrera `madvise(MADV_DONTNEED)` contra escrituras privadas (patrón Dirty COW), sin y con defensa.
`bench/bench_mmap.c`, `bench/run_bench.sh`	Micro-benchmarks y medición con perf, strace y ftrace.
`Makefile`	`make` (BTF → `vmlinux.h` → `.bpf.o` → skeleton → binarios), `make test`, `make race`, `make bench`, `make clean`.

Núcleo de la decisión (extracto de `mmap_guard.bpf.c`):

```
SEC("lsm/mmap_file")
int BPF_PROG(guard_mmap_file, struct file *file, unsigned long reqprot,
             unsigned long prot, unsigned long flags, int ret)
{
    if (ret != 0) return ret;        /* otro LSM ya denegó */
    if (!file)    return 0;          /* mapeo anónimo */
    int shared = (flags & MAP_TYPE) == MAP_SHARED ||
                 (flags & MAP_TYPE) == MAP_SHARED_VALIDATE;
```

```
    return enforce(file, prot, flags, shared, HOOK_MMAP_FILE);
}

static __always_inline int enforce(struct file *file, unsigned long prot, ...)
{
    struct file_key key = {};
    struct file_policy *pol = lookup_policy(file, &key);   /* (s_dev, i_ino) */
    if (!pol) return 0;                                    /* no protegido */
    __u32 uid    = current_euid();                         /* task->cred->euid */
    __u32 reason = violated_rule(pol->flags, shared, prot, is_mmap);
    if (reason && uid == pol->allowed_uid) reason = 0;     /* UID exento */
    ...
    emit_event(&key, uid, hook, verdict, reason, prot, flags);  /* ring buffer */
    return verdict == VERDICT_DENY ? -EPERM : 0;
}
```

Política (`policy.conf`): una línea por archivo con `<ruta> <uid_autorizado|-> <reglas>`. Las reglas son `shared-write`, `private-write`, `write`, `exec` y `all`. Decisiones de diseño:

- **Identificación por inodo y no por ruta.** Enlaces simbólicos, enlaces duros, *bind mounts* y renombrados no eluden la política. Lo comprueban las pruebas de enlace simbólico y enlace duro.
- **UID efectivo** (`cred->euid`), que es el que cuenta para permisos. **Root no está exento por defecto**: un proceso root no autorizado también es denegado.
- **Respeto de la pila LSM:** si un LSM anterior ya denegó (`ret != 0`), se propaga su decisión.
- **Modo auditoría (`-a`)**: registra lo que denegaría sin bloquear, útil para desplegar una política sin romper aplicaciones.

Compilación limpia en WSL2 (salida real de `make`):

```
clang -g -O2 -target bpf -D__TARGET_ARCH_x86 -Wall -Wextra -Wno-unused-parameter -Werror \
      -Ibuild -c src/bpf/mmap_guard.bpf.c -o build/mmap_guard.bpf.o
bpftool gen skeleton build/mmap_guard.bpf.o > build/mmap_guard.skel.h
cc -O2 -g -std=gnu11 -Wall -Wextra -Werror -Ibuild -o mmap_guard src/loader/mmap_guard.c \
      src/loader/policy.c -lbpf -lelf -lz
cc -O2 -g -std=gnu11 -Wall -Wextra -Werror -o tests/mmap_probe tests/mmap_probe.c
```

4.2 Demostración: pruebas de ataque y defensa

`make test` ejecuta 13 casos como el usuario sin privilegios `nobody` (o como root, cuando corresponde) sobre archivos de prueba con permisos 0666, de modo que **los permisos POSIX permiten todo** y cualquier denegación proviene del LSM. Política: `secreto.dat` con `write,exec` y sin UID autorizado; `config.dat` con `shared-write` y UID autorizado 0.

Caso	Usuario	Sin defensa	Con defensa	Resultado
Lectura MAP_SHARED (legítima)	nobody	permite	permite	OK
Escritura MAP_SHARED	nobody	permite	deniega	OK
Escritura MAP_PRIVATE con O_RDONLY (COW)	nobody	permite	deniega	OK
Mapeo PROT_EXEC (inyección de código)	nobody	permite	deniega	OK

Caso	Usuario	Sin defensa	Con defensa	Resultado
Bypass: mmap(R) + mprotect(W) shared	nobody	permite	deniega	OK
Bypass: mmap(R) + mprotect(W) private	nobody	permite	deniega	OK
Bypass: mmap(R) + mprotect(X)	nobody	permite	deniega	OK
Bypass: enlace simbólico	nobody	permite	deniega	OK
Bypass: enlace duro	nobody	permite	deniega	OK
Root sin autorización	root	permite	deniega	OK
Config: escritura shared no autorizada	nobody	permite	deniega	OK
Config: escritura shared por UID autorizado	root	permite	permite	OK
Config: escritura private (regla no aplica)	nobody	permite	permite	OK
Integridad: sha256 del secreto antes = después	—	—	igual	OK
Auditoría: escritura permitida y registrada (-a)	nobody	—	permite + registra	OK

Resultado: 29 de 29 verificaciones correctas en WSL2 (kernel 6.6.87.2). Los contadores del kernel en la fase de bloqueo registraron 1187 evaluaciones de gancho, 16 sobre archivos protegidos y **10 denegaciones**, exactamente los 10 casos de ataque esperados. Ejemplo de evento de auditoría:

```
08:58:42 DENEGADO  mmap  pid=3765 uid=65534 comm=mmap_probe prot=RW- flags=0x1 \
         regla=shared-write archivo=/tmp/mmap_guard_test.Zs4U7z/config.dat
```

4.3 Prueba de condición de carrera (patrón Dirty COW)

`race_cow` mapea un archivo `root:root 0444` y lanza dos hilos durante N segundos: uno ejecuta `madvise(MADV_DONTNEED)` en bucle y el otro escribe en la región privada, ya sea forzando la escritura vía `/proc/self/mem` (el vector original de CVE-2016-5195) o directamente por puntero sobre un mapeo COW. Al terminar compara el archivo, leído con `pread()`, contra una instantánea previa.

Escenario	madvise	Escrituras	Resultado
1. Sin defensa: madvise vs. /proc/self/mem (Dirty COW)	3 653 267	1 963 236	Archivo intacto (sha256 igual)
2. Sin defensa: madvise vs. escritura directa COW	4 104 992	869 691 015	Archivo intacto (sha256 igual)
3. Con defensa: madvise vs. escritura directa COW	—	—	mmap(RW, MAP_PRIVATE) denegado: EPERM
4. Con defensa: madvise vs. /proc/self/mem	3 663 258	2 068 280	Archivo intacto (lo protege el kernel)

4 de 4 escenarios correctos. Kernel 6.6.87.2-microsoft-standard-WSL2, 8 CPUs, 10 s por escenario, atacante `nobody` contra un archivo `root:root 0444` de 8 KiB. El log del guard registró la denegación: `DENEGADO mmap uid=65534 comm=race_cow prot=RW- flags=0x2 regla=private-write`.

Interpretación. El escenario 1 demuestra la mitigación del propio kernel (pregunta 9): millones de escrituras forzadas en plena carrera con `MADV_DONTNEED` y el archivo queda intacto. El escenario 3 muestra que nuestra defensa corta el ataque antes: el mapeo escribible ni siquiera se crea. El

escenario 4 muestra un límite honesto de nuestra solución: una escritura por `/proc/self/mem` no pasa por `mmap` ni `mprotect`, así que el LSM no la ve, y la integridad la sigue garantizando el kernel (ver 4.6).

4.4 Medición de impacto (overhead)

Metodología: `bench_mmap` mide con `clock_gettime(CLOCK_MONOTONIC)` en el proceso fijado a una CPU (`taskset`). Cada prueba se repite 5 veces y se reporta la **mediana**. Escenarios: **S0** sin defensa; **S1** defensa activa sobre un archivo no protegido (búsqueda fallida en el mapa); **S2** defensa activa sobre un archivo protegido con acceso permitido (búsqueda exitosa más evaluación de reglas); **S3** como S2, pero registrando cada acceso en el ring buffer (`-v`, peor caso). Archivos de 16 MiB en el page cache.

Prueba (unidad)	S0 sin defensa	S1 no protegido	S2 protegido	S3 protegido + log
mmap + munmap (ns/par)	2433,75	2503,20 (+2,9%)	2760,86 (+13,4%)	2916,37 (+19,8%)
mprotect (ns/llamada)	431,56	509,39 (+18,0%)	626,55 (+45,2%)	705,64 (+63,5%)
Fallo de página (ns/página)	227,46	219,02 (−3,7%)	246,80 (+8,5%)	252,49 (+11,0%)
read() 4 KiB (ns/página)	943,77	920,51 (−2,5%)	988,36 (+4,7%)	977,50 (+3,6%)
mmap + recorrido (ns/página)	597,75	601,74 (+0,7%)	629,87 (+5,4%)	571,74 (−4,4%)

Valores en ns por operación (mediana); entre paréntesis, la variación frente a S0.

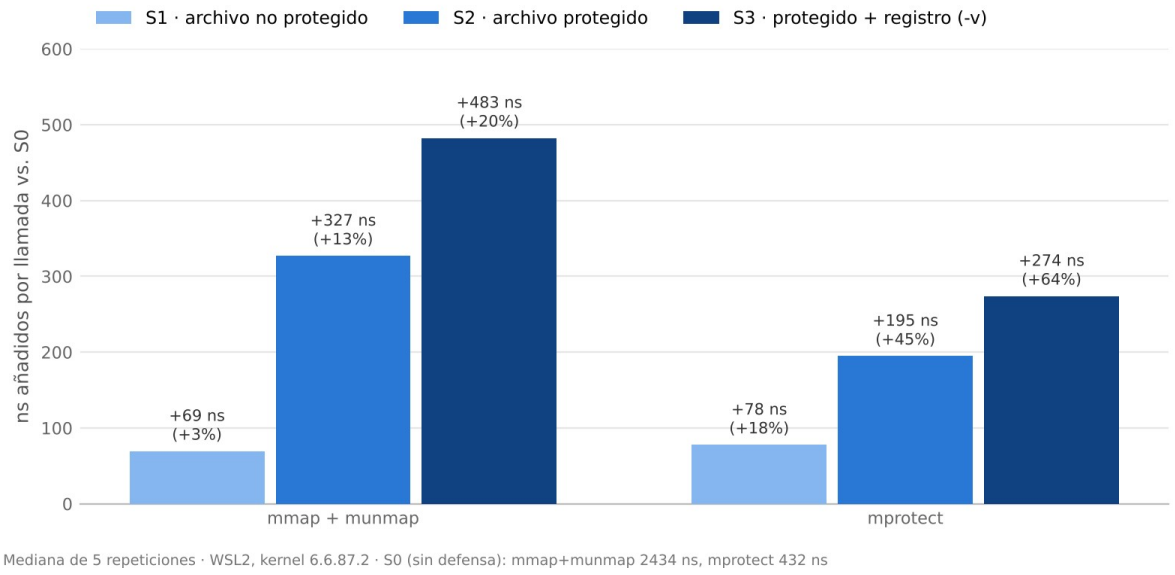


Figura 3. Costo añadido por llamada en las dos operaciones que pasan por los ganchos (mmap y mprotect), frente a S0 sin defensa.

Dónde está el costo. Solo `mmap` y `mprotect` pasan por nuestros ganchos, y ahí aparece el costo: **+69 ns por llamada** cuando el archivo no está protegido (una búsqueda fallida en el mapa hash) y **+327 ns** cuando sí lo está (búsqueda exitosa, lectura de `task->cred->euid`, evaluación de reglas y contadores por CPU). Registrar cada acceso en el ring buffer (S3) suma unos 155 ns más. En `mprotect` el porcentaje es mayor (+18% a +64%) porque la llamada base es más barata (432 ns), aunque el costo absoluto es similar (+78 a +274 ns).

Lo que no cambia. Los fallos de página, `read()` y el recorrido del mapeo no pasan por la defensa. En la prueba de fallos se hace un `mmap` por cada 4096 páginas, así que el gancho aporta menos de 0,1 ns por página. Las variaciones observadas (−4% a +11%) están dentro del ruido de las repeticiones de la línea base en WSL2: en S0, la prueba de fallos osciló entre 211 y 247 ns/página y `mmap` entre 2331 y 4622 ns. Además, S2 y S3 usan otro archivo de 16 MiB, con otra ubicación física en el page cache. Lo confirma `perf`: **exactamente 25 997 fallos de página** con y sin defensa.

Ruta caliente frente a ruta fría. El fallo menor promedio cuesta 227 ns por página, pero gracias a *fault-around* solo hay un fallo cada 16 páginas (256 por recorrido de 16 MiB). Una vez instalado el mapeo, recorrerlo cuesta 598 ns por página, frente a 944 ns con `read()` en bloques de 4 KiB (**37% menos**), que es justamente la ventaja de `mmap` que motiva el problema. La defensa preserva esa ventaja porque solo actúa al crear o modificar la VMA.

Herramientas de sistema

ftrace: el script intentó medir la duración de `security_mmap_file` con `function_graph` filtrado por PID, pero en el kernel de WSL2 el búfer no registró eventos. No alcanzamos a diagnosticar la causa antes de la entrega, así que la medición del gancho se basa en la diferencia de tiempos S1/S2 frente a S0, que aísla el mismo costo. Los contadores del propio programa (`stats_map`) confirmaron 6 105 310 evaluaciones del gancho durante el benchmark (3 061 135 sobre el archivo protegido).

perf stat (prueba de fallos, 100 recorridos de 16 MiB): S0 → 25 997 fallos de página (25 996 menores, 1 mayor), 13 cambios de contexto, task-clock 89,45 ms; S2 → **25 997 fallos de página** (idénticos), 12 cambios de contexto, task-clock 93,48 ms (+4,5%, dentro del ruido). La defensa no añade fallos ni cambios de contexto: no hay transiciones adicionales a Ring 0 más allá de la propia syscall. En WSL2 solo hay eventos de software de perf; la máquina virtual no expone los contadores de hardware (ciclos, instrucciones).

strace -c (20 000 pares mmap/munmap): S0 → `mmap` 20 µs/llamada, `munmap` 19 µs/llamada; S2 → `mmap` 22 µs/llamada, `munmap` 19 µs/llamada. strace multiplica por ~8 el costo de cada syscall (dos paradas por ptrace en cada una), así que los valores absolutos no son representativos, pero sí la distribución: el aumento aparece solo en `mmap` (la llamada con gancho) y `munmap` no cambia.

4.5 Justificación teórica: ¿por qué en la capa de kernel?

- **No es eludible desde Ring 3.** Una biblioteca `LD_PRELOAD` se evita con un binario estático, con la instrucción `syscall` directa o simplemente sin la variable de entorno. El gancho LSM está en `vm_mmap_pgoff()`, por donde pasan **todas** las rutas que crean mapeos de archivo: la syscall `mmap`, el cargador ELF de `execve` y `shmat`.
- **Decide sobre objetos ya resueltos.** Recibe el `struct file *` y la `vm_area_struct *`, no un fd o una ruta que el atacante pueda cambiar entre la verificación y el uso (TOCTOU).
- **Actúa en el único punto de control posible.** Después del primer fallo, los accesos los resuelve la MMU sin entrar al kernel (pregunta 1). Los bits R/W y NX de cada PTE salen de la VMA, así que controlar `mmap` y `mprotect` equivale a controlar el hardware de todas las páginas futuras.
- **Costo nulo en la ruta caliente.** No hay instrumentación por fallo ni por acceso: el costo se paga una vez por `mmap` o `mprotect` (sección 4.4) y no en cada fallo de página.

- **Seguridad de la propia defensa.** El verificador eBPF garantiza que el programa no puede colgar ni corromper el kernel (ningún kernel panic en todas las pruebas). Se carga y descarga sin recompilar el kernel ni cargar módulos sin firmar.

4.6 Limitaciones y trabajo futuro

Vector aún descubierto	Por qué	Mitigación propuesta
Escritura forzada vía `/proc/<pid>/mem` o `ptrace`	Usa GUP con FOLL_FORCE y no pasa por `mmap`/`mprotect`. Hoy el kernel impide que llegue al archivo (sección 4.3), pero sí altera la vista del proceso.	Ganchos `ptrace_access_check` y `file_open` sobre `/proc/*/mem`; Yama `ptrace_scope=2`.
Ruta `read()`/`write()` sin mmap	El alcance del parcial es mmap.	Añadir `lsm/file_permission` y `lsm/file_open` con la misma política por inodo.
Root puede matar el loader o descargar el programa	La protección vive mientras el loader corre.	Fijar el programa y sus mapas en bpffs (`bpf_link__pin`), proteger los mapas con `lsm/bpf`, usar *lockdown* y BPF firmado.
Reemplazo del archivo (nuevo inodo)	Un `rename()` sobre la ruta deja el nuevo inodo fuera de la política.	Ganchos `inode_rename`/`inode_unlink` o política por ruta con `bpf_d_path()`, y recarga automática.
Contenedores / user namespaces	Se compara el euid del espacio de nombres inicial.	Traducir con `from_kuid()` o aplicar la política por cgroup (`bpf_get_current_cgroup_id`).
Sistemas de archivos sin inodos estables (9p/drvfs de WSL, overlayfs)	El inodo visible puede no coincidir con el del archivo real.	Resolver el inodo real (`file_inode` del archivo de respaldo) o proteger por ruta.
Sin inspección ni cifrado del contenido	Se decide por VMA, no por página.	fscrypt para el cifrado en reposo; instrumentar `page_mkwrite` para auditar escrituras.
Auditoría con pérdida bajo carga extrema	En el modo `-v`, con unos 3 millones de `mmap` en segundos, el ring buffer de 256 KiB se saturó: solo se entregaron 45 111 eventos. Por diseño se descarta el evento antes que bloquear la syscall.	Contar los descartes en `stats_map`, agregar en el kernel (contadores por archivo/UID), muestrear y usar un buffer mayor. Las **denegaciones** nunca se pierden: la decisión no depende del registro.
Kernel comprometido	Un atacante con ejecución en Ring 0 puede desactivar cualquier LSM.	Protección a nivel hipervisor (abajo).

Propuestas de mitigación a nivel de hardware e hipervisor:

- **Intel MPK/PKU** (`pkey_mprotect`, registro PKRU): aislar regiones sensibles dentro del propio proceso y revocar su acceso sin cambiar las tablas de páginas.
- **Protección del kernel por el hipervisor** (Hyper-V VBS/HVCI, que WSL2 ya usa como plataforma; también KVM con EPT): las tablas de segundo nivel (EPT/NPT) marcan el código del kernel y las estructuras críticas como no escribibles incluso para Ring 0, de modo que un kernel comprometido no puede desactivar el LSM.
- **Cifrado de memoria** (AMD SEV-SNP, Intel TDX/TME): el texto claro del page cache queda cifrado en la RAM frente a ataques físicos y frente al propio hipervisor.

- **Intel CET** (*shadow stack*, IBT) y **Arm MTE/BTI**: dificultan explotar una inyección aunque llegue a producirse.
- **IOMMU**: impide que dispositivos con DMA escriban en las páginas del page cache saltándose la MMU.

5. Conclusiones

- El control de seguridad sobre archivos mapeados debe ejercerse **al crear y modificar la VMA**. Una vez instalada la PTE, el hardware atiende los accesos y ni el espacio de usuario ni el propio kernel los vuelven a ver.
- Enganchar la tabla de syscalls dejó de ser viable en kernels 6.x (símbolos no exportados, CR0 pinning, despacho por `switch` en 6.9). BPF LSM ofrece la cobertura de un LSM con la flexibilidad de un módulo y la seguridad del verificador.
- Identificar los archivos por inodo y controlar también `mprotect` cierra las evasiones más comunes (enlaces y escalada W/X), como mostraron las pruebas diferenciales sin y con defensa.
- El kernel moderno ya mitiga la carrera de Dirty COW (`PageAnonExclusive`). Nuestra defensa añade una capa previa que impide crear los mapeos escribibles, en lugar de depender solo de la corrección de GUP.
- El costo de la defensa se concentra en las llamadas `mmap`/`mprotect` y es nulo en la ruta de fallos de página. En WSL2 medimos +69 a +327 ns por mmap (3–13%) y +78 a +195 ns por mprotect; los fallos de página y read() quedaron dentro del ruido de medición.
- La IA aceleró la exploración, la generación de código y el diseño de pruebas, pero seis errores o fallos técnicos reales (H1–H6) solo se detectaron al compilar, ejecutar en el kernel real y contrastar con el código fuente. La validación humana sigue siendo indispensable.

6. Referencias

1. Linux kernel 6.6, código fuente: `mm/mmap.c`, `mm/util.c`, `mm/memory.c`, `mm/gup.c`, `mm/madvise.c`, `arch/x86/mm/fault.c`, `include/linux/lsm_hook_defs.h`, `security/security.c`, `kernel/trace/bpf_trace.c`.
2. Documentación del kernel: "LSM BPF Programs" (docs.kernel.org/bpf/prog_lsm.html); "Filesystem-level encryption (fscrypt)" (docs.kernel.org/filesystems/fscrypt.html); "Linux Security Module Development" (docs.kernel.org/security/lsm-development.html).
3. CVE-2016-5195 (Dirty COW), NVD; parche 19be0eaffa3a "mm: remove gup_flags FOLL_WRITE games from __get_user_pages()".
4. D. Hildenbrand, "mm/gup: fix FOLL_FORCE COW security issue and remove FOLL_COW", Linux 6.0 (2022).
5. CVE-2017-1000405 (Huge Dirty COW) y CVE-2022-0847 (Dirty Pipe).
6. libbpf y bpftool (github.com/libbpf), libbpf 1.3; "BPF CO-RE reference guide", A. Nakryiko.
7. Intel 64 and IA-32 Architectures Software Developer's Manual, Vol. 3A, cap. 4 (Paging) y 5 (Protection).
8. Universidad EAFIT, SO2026B, repositorio del curso: github.com/evalenciEAFIT/SO2026B (USOS/CapaKernel).

Anexo A. Reproducción de los resultados

Requisitos: WSL2 (o Linux) con kernel ≥ 5.7, `CONFIG_BPF_LSM=y`, BTF y `bpf` en `lsm=` (en WSL2: `docs/entorno/.wslconfig` y luego `wsl --shutdown`). Paquetes de Ubuntu 24.04: `clang llvm libbpf-dev libelf-dev zlib1g-dev build-essential linux-tools-generic strace`.

```
cat /sys/kernel/security/lsm        # debe incluir ,bpf
make                                # clean build
make test                           # 13 ataques x (sin/con defensa) + integridad + auditoría
make race                           # carrera madvise vs. escrituras (4 escenarios)
make bench                          # overhead: mediana de 5 repeticiones, perf, strace,
ftrace
sudo ./mmap_guard -c policy.conf -s # uso interactivo (Ctrl+C para salir)
```

Los reportes quedan en `results/` (`test_report.txt`, `race_report.txt`, `bench/summary.csv`, `bench/ftrace_summary.csv`, `bench/perf_*.csv`, `bench/strace_*.txt` y `audit_tests.jsonl`).